

Assignment 1B: Agentic Foundations

CSCI-GA.2630 · Foundations of Networks and Mobile Systems

"Imagination is the beginning of creation. You imagine what you desire, you will what you imagine and at last you create what you will." ~ George Bernard Shaw

Throughout your careers, you young fragments of brilliant light will be creating systems, tools, and agents to change the world. Crystalize and sharpen your imagination and use it to forge tools to make what was once impossible now a necessary part of human society. Netflix imagined a world to stream everything, Apple imagined a world to empower people with knowledge through a device, and Tesla imagined a world where everything runs on electricity. How will you use your imagination to forge and redesign society as we know it?

To begin this odyssey to do the impossible, let's first build a simple agent to prepare you for what is to come. You will be creating an agent to complete a task or objective; an execution of your will and desire.

The Task

Build an agentic tracker: an agent that finds the top K developments in a topic or target you care about, ranks them, and summarizes them with sources. When you run it again, it reports what's new since the last run.

This is the web research agent from lecture (search → fetch → observe → decide → synthesize), with memory added. The lecture agent answered one question and stopped. Yours has to remember what it has already seen. Its results appear in the app you built in A1, which makes this the first tool in your command center.

You choose the topic and K, where $3 \leq K \leq 10$. Pick a narrow topic or target. Each of these makes a better tracker than "AI":

- "open-source robotics foundation models"
- "launches and pricing changes at three competitors to your startup idea"
- "new ML engineering roles at ten companies you'd apply to"

Track only sources you're allowed to fetch. Career pages and public job-board APIs are fine. Scraping sites behind a login, or sites whose terms prohibit it, is not.

Before you start

Check your A1 database. Your A1 login has to work for us to see your display.

- If you used Render Postgres, migrate before you start. Free instances are deleted 30 days after creation.

- If you used Supabase, free projects pause after a week idle. Make sure yours is awake before grading.

Set a hard spend cap on every paid key before your first request. Also check each provider's daily cap, not just its per-minute limit, because a tracker multiplies calls by the number of runs.

Requirements

1. You write the agent loop yourself. You may use model SDKs and HTTP libraries. You may not use an agent framework for the loop (LangChain agents, CrewAI, OpenClaw, and similar). Validation, budgets, and state are what's being graded, and a framework hides all three.
2. Tools. Provide at least `search_web(query)`, `fetch_article(url)`, and `finish(report)`. Each tool must be callable without the model: `python -m tracker.tools fetch_article <url>`.
3. Policy lives in `config.yaml`. It holds the topic, K, model, instructions, tools, limits (`max_steps`, `max_fetches`, and a token or cost budget), and the allowed schemes and hosts. This follows the lecture's configuration slide.
4. The runtime enforces budgets. The lecture's minimal loop is deliberately incomplete, and a bare `while True` fails this requirement. When a budget runs out, the agent stops and writes a report marked partial, using the evidence it has so far.
5. Classify failures. Retry transient failures (timeouts, per-minute rate limits) with backoff and a cap on retries. Stop with a clear message on terminal failures (bad key, exhausted daily quota, payment required). Retrying a daily quota is a bug.
6. Recrawl. Persist state between runs, in SQLite, a JSON file, or your A1 database (see the optional section). At minimum, store:
 - which URLs you have already fetched;
 - which developments you have already reported, and which sources support each one;
 - what your top K was last time.
7. On the second run, the agent must:
 - skip articles it already has;
 - recognize when a new URL reports a development it has already covered;
 - produce a report organized as New since last run, then Still in top K, then Dropped.
8. Guardrail `fetch_article`. Before making any request:
 - reject any scheme other than `http(s)`;
 - reject hosts that resolve to loopback, private, or link-local addresses;
 - enforce a timeout and a size limit.
9. Retrieved web text is data. It must not be able to change the agent's instructions, tools, or budget.
10. Provenance. Every development in the report cites its source. A claim that doesn't appear in the cited source counts as a hallucination.
11. Display in your A1 app. Add a screen to your A1 frontend, behind your A1 login, that shows:

- the latest report: ranked developments, each with its summary and source links;
 - run history: when each run happened and what changed;
 - the articles fetched on each run, with title, URL, fetch time, and status (fetched, skipped as already seen, or rejected by your guardrail).
12. The frontend gets this data from new endpoints on your A1 backend. Those endpoints can read from wherever your tracker stores its state. A1's rules apply to them: a missing or invalid token returns 401. Render anything that came from the web as text, never as HTML. The pages your agent fetches are untrusted, and the injection page we seed will contain markup and script. If that script runs in your app, you have a stored-XSS bug.
 13. Trace log. Log every model call and tool call with step, tool, arguments, status, latency, and tokens or credits.
 14. Secrets. A1's single exception still applies: you may include the connection string for your throwaway course database. Everything else (model, search, and fetch keys) goes in environment variables, with a `.env.example` listing them.

(OPTIONAL) Store your tracker's data in your A1 database

These steps are suggestions, not rules. The required part is Recrawl: your tracker must remember what it has seen between runs. This section is one way to do that, using the system you built in A1.

1. Decide what your tracker needs to save and load. Start with the data listed under Recrawl. Save anything else you find useful, such as run timestamps, token counts, or summaries.
2. Design your schemas around that data. Create new tables if you use SQL, or new collections if you use MongoDB. There is no required schema. Choose the structure that makes your recrawl simple, and be ready to explain your choice.
3. Add API endpoints to your A1 backend that store data (writing results at the end of a run) and fetch it (loading history at the start of the next run). A1's rules apply to all of them:
 - A missing or invalid token returns 401.
 - One user can't read or change another user's tracker data.
4. Have your agent save and load its history only through your API. It should never connect to the database directly. That makes your tracker a client of the backend you built.
5. Document it. In your README, show your schema and list your new endpoints in the same table format as A1.

Submission

Submit the repo URL in Brightspace. The repo lives in the class org and contains:

- the tracker code and config.yaml
- your updated A1 backend and frontend, in the same repo or linked from the README
- a README that meets A1's requirements (prerequisites, exact commands in order, a .env.example, anything unusual), with these commands listed in order:
 - bring up your A1 backend and frontend
 - run the tracker
 - run it again
 - reset its saved state
- reports/run1.md and reports/run2.md, from runs at least a day apart
- the trace logs for both runs

AGENT.md

1. **Workflow vs. agent.** Which decisions does the model make, and which does your code make? Describe one decision you moved out of the model, and say why.
2. **The network.** Using your trace, how many round trips did run 1 take, and to which services? Where did the time go?
3. **"New."** How do you decide that two articles describe the same development? Give one case your method gets wrong.
4. **Failure.** A 429 comes back. What does your code do? Quote it. How does its behavior differ for a per-minute limit versus a daily cap?
5. **Budget.** What does one run cost? If you ran daily, which free tier would run out first, and on what day?

Grading: 25 pts

	Points
Runs from a clean clone; produces a top-K report on your topic	4
Recrawl: uses saved state, skips cached articles, separates new from already-seen	4
Budgets enforced; transient and terminal failures handled correctly (tested with a bogus key and with the network cut)	5
Guardrails: we call <code>fetch_article</code> directly with bad URLs and seed an injection page	4

Provenance: we check two claims against their cited sources	2
AGENT.md is specific to your system	5
No committed secrets	1

TOOLS

Free API
https://github.com/public-apis/public-apis?tab=readme-ov-file#music

Web Extraction

FireCrawl	Site: https://www.firecrawl.dev Docs: https://docs.firecrawl.dev Self-hosted Code: https://github.com/firecrawl/firecrawl
-----------	---

Web Search

Tavily is search built for agents, with search, extract, crawl and map endpoints. It reportedly has 1,000 free credits a month with no card, though that figure comes from a competitor's blog, not Tavily's own page.	
Tavily	Site: https://www.tavily.com Docs: https://docs.tavily.com

Free Large Language Models (Recommended)

Groq	Get an API key: https://console.groq.com/keys Free-tier limits: https://console.groq.com/docs/rate-limits Quickstart: https://console.groq.com/docs/quickstart
Google Gemini	Get an API key: https://aistudio.google.com/api-keys Rate limits: https://ai.google.dev/gemini-api/docs/rate-limits Billing (read before enabling it): https://ai.google.dev/gemini-api/docs/billing
OpenRouter	Site: https://openrouter.ai Limits: https://openrouter.ai/docs/api-reference/limits
Others	Qwen , available in small sizes: https://ollama.com/library/qwen3 Llama 3.2 , 1B and 3B: https://ollama.com/library/llama3.2 Gemma 3 , 1B and 4B: https://ollama.com/library/gemma3 Phi-4-mini (Microsoft): https://ollama.com/library/phi4-mini DeepSeek-R1 , small distilled versions: https://ollama.com/library/deepseek-r1

Open Source Large Language Models (use at own discretion)

These can consume a lot of memory and RAM. Unless you are willing to experiment and risk this, please be wary about using open source models as it can be incredibly hungry for memory and computation.

Ollama	Download: https://ollama.com/download Instructions: https://github.com/ollama/ollama Model library: https://ollama.com/library
LM Studio (option for students who want a GUI instead of a terminal)	https://github.com/ggml-org/llama.cpp
llama.cpp	https://github.com/ggml-org/llama.cpp

Priced Large Language Models (use at your own discretion)

These models are useful, but at a cost. Use them carefully and it can easily consume your tokens if parameters are not carefully designed.

Anthropic (Claude)	API keys and console: https://platform.claude.com Pricing: https://claude.com/pricing#api Docs: https://docs.claude.com
Google Gemini	API keys: https://aistudio.google.com/api-keys Pricing: https://ai.google.dev/gemini-api/docs/pricing Billing setup: https://ai.google.dev/gemini-api/docs/billing
DeepSeek	API keys and console: https://platform.deepseek.com Pricing: https://api-docs.deepseek.com/quick_start/pricing
OpenAI	API keys: https://platform.openai.com/api-keys Pricing: https://openai.com/api/pricing